



Módulo 2: La primera página (HomeController)

El patrón MVC

Spring MVC organiza la aplicación en tres capas:

- **Model** — los datos que queremos mostrar
- **View** — la plantilla HTML que los renderiza
- **Controller** — recibe la petición HTTP, prepara los datos y decide qué vista mostrar

El flujo de una petición es siempre el mismo:

Navegador → Controller → (Model) → View → HTML al navegador

Creamos HomeController

Dentro del paquete `controller`, creado dentro del paquete base, creamos la clase:

```
package com.unaempresa.primerproyecto.controller; import
org.springframework.stereotype.Controller; import
org.springframework.web.bind.annotation.GetMapping; @Controller public
class HomeController { @GetMapping({"/home", "/", ""}) public String
home() { return "home/home"; } }
```

- `@Controller`
 - Le dice a Spring que esta clase es un controlador MVC.
 - Gracias al `@ComponentScan` de la clase principal, Spring la encuentra automáticamente al arrancar.
 - Devuelve el nombre de una vista para que Thymeleaf la renderice.
- `@GetMapping` : Asoció una o varias URLs HTTP GET a este método. Cuando el navegador pida cualquiera de estas tres rutas, Spring ejecutará el método `home()` :
 - `http://localhost:8080/`
 - `http://localhost:8080/home`
 - `http://localhost:8080` (cadena vacía)

El valor de retorno

El método devuelve el `String` `"home/home"` . Thymeleaf interpreta esto como:

```
src/main/resources/templates/home/home.html
```

El valor de retorno es la ruta relativa dentro de `templates/` , sin la extensión `.html` .

Creamos la carpeta y el template

Dentro de `src/main/resources/templates/` creamos la carpeta `home` y dentro el fichero `home.html` :

```
<!DOCTYPE html> <html xmlns:th="http://www.thymeleaf.org"> <head> <meta
charset="UTF-8"> <title>Home</title> </head> <body> <h1>Home</h1>  <br /> <a
href="https://www.google.com">Ir a google</a> <br /> <a
th:href="@{/animales/lista}">Ir a la lista de animales</a> <br /> <a
th:href="@{/contadores/lista}">Ir a la lista de contadores</a> </body>
</html>
```

El namespace de Thymeleaf

```
<html xmlns:th="http://www.thymeleaf.org">
```

Esta declaración en la etiqueta `<html>` le dice al editor que los atributos `th:*` son válidos y pertenecen a Thymeleaf. Sin ella el código funciona igual, pero el IDE marcará todos los `th:` como errores de validación HTML y perderemos el autocompletado.

Regla: **siempre** añadir `xmlns:th="http://www.thymeleaf.org"` en la etiqueta `<html>` de todos los templates.

Recursos estáticos

Las imágenes, CSS y JavaScript se colocan en `src/main/resources/static/`. Spring Boot los sirve automáticamente desde la raíz de la aplicación.

El logo de Google lo colocamos en:

```
src/main/resources/static/img/googlelogo.png
```



Para cargarlo en el template usamos `th:src` con la sintaxis `@{...}` :

```

```

Thymeleaf resuelve `@{img/googlelogo.png}` generando la URL correcta relativa al contexto de la aplicación.

Nota sobre el atributo `width` : el atributo HTML `width` en `<img>` acepta solo números, sin unidades. `width="200"` es correcto; `width="200px"` también funciona en la mayoría de navegadores pero no es válido según el estándar HTML.

Enlace externo vs enlace interno

Hay dos formas de poner un enlace en Thymeleaf y es importante distinguirlas:

```
<!-- Enlace externo: URL absoluta, usamos href normal --> <a  
href="https://www.google.com">Ir a google</a> <!-- Enlace interno:  
ruta de nuestra app, usamos th:href con @{ } --> <a  
th:href="@{/animales/lista}">Ir a la lista de animales</a>
```

¿Por qué `th:href` para los internos?

Si la aplicación se despliega en un contexto con prefijo (por ejemplo `https://servidor/miapp/animales/lista`), Thymeleaf ajusta la URL automáticamente. Con un `href` normal la URL quedaría rota. Además, `@{...}` permite construir URLs dinámicas con variables, como veremos más adelante.

Probamos

Arranca la aplicación y abre `http://localhost:8080` . Deberías ver:

- El título `Home`
- El logo de Google
- Los tres enlaces (los dos internos darán 404 por ahora, aún no existen esas rutas)